

Practical 4

AIM: To Implementation and Time analysis of factorial program using iterative and recursive method

Algorithm:

1. Factorial Using Iterative Method

Input: A non-negative integer n

Output: Factorial of $n!$

Algorithm: Factorial using Iterative Method

Step-1: Start

Step-2: Read the value of n .

Step-3: Set $fact = 1$.

Step-4: For $i = 1$ to n , repeat the following:

$fact = fact \times i$

Step-5: Display $fact$.

Step-6: Stop.

2. Factorial Using Recursive Method

Input: A non-negative integer n

Output: Factorial of $n!$

Algorithm: Factorial using Recursive Method

Step-1: Start

Step-2: Read the value of n .

Step-3: If $n = 0$ or $n = 1$, return 1.

Step-4: Otherwise, calculate:

$factorial(n) = n \times factorial(n - 1)$

Step-5: Return the calculated value.

Step-6: Display the factorial.

Step-7: Stop.

Program:

```
//iterative Method
#include <iostream>
using namespace std;
class Factorial
```

```
{
public:
int fact(int n)
{
int result = 1;
for (int i = 1; i <= n; i++)
{
result = result * i;
}
return result;
}
};
int main()
{
int n,result;
Factorial obj;
cout << "Enter the value to find its factorial: ";
cin >> n;
result = obj.fact(n);
cout << "Factorial of " << n << " is " << result << endl;
return 0;
}
//Recursive method
#include <iostream>
using namespace std;
class Factorial
{
public:
int factr(int n)
{
if (n == 0 || n == 1)
{
return 1;
}
else
{
int result = n * factr(n - 1);
return result;
}
}
};
int main()
{
int n, result;
// Create an object of the class
Factorial obj;
cout << "Enter the value to find its factorial: ";
```

```
cin >> n;  
result = obj.factr(n);  
cout << "Factorial of " << n << " is " << result << endl;  
return 0;  
}
```

Output:

```
ENTER THE NUMBER TO GET FACTORIAL:5  
ENTER 1 FOR ITERATIVE METHOD 2 FOR RECURSION METHOD :1  
FACTORIAL IS: 120
```

```
ENTER THE NUMBER TO GET FACTORIAL:5  
ENTER 1 FOR ITERATIVE METHOD 2 FOR RECURSION METHOD :2  
120
```

Time Complexity:

time complexity

1. Iterative Method

$fact = 1 \rightarrow 1$ operation

$int\ i = 1 \rightarrow 1$ operation

Loop condition $for(int\ i=1; int\ i \leq n; i++) \rightarrow n + 1$ operations

$fact = fact * i \rightarrow n$ operations

$i++ \rightarrow n$ operations

Therefore,

$$T(n) = 1 + 1 + (n + 1) + n + n$$

$$T(n) = 3n + 3$$

Ignoring constants and lower-order terms:

$$T(n) = O(n)$$

2. Recursive Method (Recursive factorial)

```
int factorial(int n){
```

```
if (n == 0 || n == 1)
```

```
return 1;
```

```
return n * factorial(n - 1);}
```

$$T(n) = T(n - 1) + O(1)$$

Expanding:

$$T(n) = T(n - 2) + O(1) + O(1)$$

$$T(n) = T(n - 3) + O(1) + O(1) + O(1)$$

Continuing until $n = 1$:

$$T(n) = T(1) + (n - 1)O(1)$$

Therefore:

$$T(n) = O(n)$$

Conclusion:

The factorial was successfully implemented using iterative and recursive methods. Both methods have $O(n)$ time complexity. The iterative method uses $O(1)$ space, while the recursive method uses $O(n)$ space.